

Detection modelling

What a sequence model adds to the Wazuh lab, measured on 2.6 million real alerts, and what it does not add

Step 5 of the Wazuh and MITRE ATT&CK lab · 2026-09-22

Step 4 of this lab detects single events. Rule 100110 counts one failed SSH password; rule 100112 notices an account being created. Neither knows the other exists, and neither knows what an ordinary Tuesday looks like on the endpoint. This step is the attempt to add the thing they cannot do, which is to read a run of alerts and judge it.

The result is that a sequence model does not earn its place here. Measured on 2,600,263 real alerts from eight independent networks, plain logistic regression on rule counts and timing beats a GRU over the alert sequence on 7 held out networks out of 8, by a mean of 0.063 average precision with a spread of 0.048 on that margin. It is the only comparison in this document consistent enough to carry a verdict. On this evidence PyTorch is not justified for this problem.

The model that is deployed on the lab dashboard is therefore a logistic regression over eleven features, and it is deployed with its measurement attached rather than as a detection.

1. Why a model was tried at all

The rules in step 4 are good at what they do and structurally blind to sequence. A brute force attempt followed four minutes later by a new account and a cron entry is three separate alerts to a Wazuh manager, and an analyst reading a queue has to assemble that story by hand. The question worth asking is whether something that reads the window rather than the event can order that queue better than the rules already do.

That question has a clear failure condition, and stating it first is the point. A model that cannot beat the single best rule is not adding anything, and it is easy to build one that looks impressive and is not. The comparison below is arranged so that a bad result is visible rather than avoidable.

2. The dataset

Collecting a labelled dataset from this lab means a 14 hour campaign, and public data was checked first. Kaggle and Hugging Face were both searched and neither has anything usable for this problem: what is there is network flow tables, which are the wrong telemetry, or small instruction tuning sets, or synthetic data under a licence that forbids redistribution. There is one public dataset of real Wazuh alerts with ground truth labels.

The **AIT Alert Data Set** (Zenodo record 8263181, CC-BY-4.0, published with the CSET 2024 paper) is alerts from eight simulated enterprise networks, each carrying a labelled multi-step intrusion. The alerts are in native `alerts.json` form, which is the same record this lab's own manager writes. The import read 2,600,263 alert records out of the archive.

What this dataset cannot answer, stated plainly. It is not a test of this lab's rules. None of rules 100100 to 100113 have a parent signature anywhere in it: no sshd brute force in any of the eight scenarios, no local account creation, no file integrity monitoring. Rules 5501 and 5502 are the entire overlap, and what AIT calls password cracking is offline hash cracking and WPScan rather than SSH. So everything below measures the **method**. Only a campaign on this lab measures the lab, and that is section 9.

NETWORK	ALERTS READ	EPISODES	ATTACK	RATE
fox	462,523	994	28	2.8%
harrison	583,754	1,391	45	3.2%
russellmitchell	41,488	766	20	2.6%
santos	126,513	1,026	21	2.0%
shaw	68,539	1,222	21	1.7%
wardbeck	88,204	1,076	14	1.3%
wheeler	603,939	1,181	15	1.3%
wilson	625,303	1,276	24	1.9%

Windowed into 300 second tumbling episodes, that is **8,932 episodes, 188 of them attack**, a base rate of 2.1 %, across 31 distinct signatures. Median 72 alerts per episode, 90th percentile 241, capped at 4096.

How an episode is built

An episode is a fixed 300 second window of alerts, labelled attack if it overlaps a ground truth attack phase from the dataset's own `labels.csv`. Tumbling rather than sliding, so no alert is counted twice and no two episodes share evidence. Boundaries fall on the epoch rather than on the first alert of the capture, because that is where the live scorer puts them and a training window has to be the window the model will be shown. The tail of an over-long window is kept rather than the head, because persistence lands at the end of a window, and the cap is set to the live ceiling rather than to a round number.

How much of this is the window, and how much is the split

Both are choices, so both were measured. The full logistic, same folds, at three widths:

WINDOW, SECONDS	AVERAGE PRECISION	SD
120	0.209	0.109
300	0.251	0.108
600	0.319	0.092

Width matters, and more than an earlier version of this document said. Average precision runs from 0.209 to 0.319 across that range, rising with the window: a wider window holds more of an attack phase and ranks it better. The earlier flat reading was an artefact of a per-window alert cap of 256, which discarded the extra alerts a wider window collected before the features ever saw them. Any figure here should be read with its width attached. 300 seconds is deployed because it is what the panel buckets at, not because it scores best.

The split matters less, and in the safe direction. Held out by time rather than by network, the same model scores 0.314 against 0.251 on the network split. A time split shares networks between fitting and test, so it

should be the easier of the two, and it is. Had it come back much lower, the network folds would have been measuring something other than generalisation.

3. The training process

Four decisions do more work than the model choice, and all four are arranged to make a flattering result harder to obtain.

Whole networks are held out, never a random sample

Episodes from one capture are a time series. Shuffling before splitting lets the model see the future: the test set would sit between training examples drawn minutes either side of it, and the score would improve for a reason that has nothing to do with detection. Each AIT scenario is a separate network, so whole networks are held out instead. A model that scores well on a network it has never seen has generalised. A model that scores well on the last hour of a network it trained on may only have learned that network's noise floor.

Eight folds, not one split

Any single network here would have supported a different conclusion. The logistic model beats the GRU by 0.137 average precision on one of them and loses to it by 0.002 on another; it beats the deployed portable feature set by 0.088 on one and loses to it on 3 of the 8. Which network is held out moves the answer further than which model is fitted, so a result taken from one split is a result about that split. Every result below is leave one network out across all 8 networks: train on seven, test on the eighth, repeat. Counting how many folds each model wins is the claim that survives.

Average precision, not f1 at a fixed cutoff

At a 2.1 % base rate, f1 at 0.5 says more about where a model happens to put its threshold than about whether it ranks attacks above background. The same logistic predictions score f1 0.162 at 0.5 and 0.317 at a threshold chosen on validation. Average precision is the area under the precision-recall curve and cannot be flattered that way. Its floor is the base rate: a model ranking at random scores 0.021 here.

Nothing is chosen on the test set

The vocabulary of known signatures is built from the fitting sources only, so a signature that appears solely in the held out network falls through to an unknown index, which is the correct thing for it to do. Thresholds and early stopping are chosen on a second whole network, held back from fitting inside the seven and rotated so that each one validates exactly once. Standardisation and the vocabulary are fitted on the remaining six alone, not on the seven, so nothing the operating point was chosen against contributed to the weights either. The GRU is run with three seeds per fold and the mean reported, because a single seed on 188 positives is noise.

4. Every model tried, and what it scored

Leave one network out, 8 folds, mean and standard deviation across them.

MODEL	F1	SD	AVERAGE PRECISION	SD
Best single rule	0.235	0.148	0.160	0.098
Logistic, portable features (deployed)	0.227	0.114	0.220	0.099
GRU over the sequence (3 seeds a fold)	0.164	0.123	0.188	0.085
Logistic, full features	0.251	0.154	0.251	0.108
Random, for reference	—	—	0.021	—

A fifth model, the degenerate classifier that calls everything an attack, is not in the table because it exists only to show what recall alone is worth. It catches every intrusion and is useless.

Average precision per fold

HELD OUT	EPISODES	ATTACK	BEST RULE	GRU	PORTABLE	LOGISTIC
fox	994	28	0.202	0.254	0.255	0.253
harrison	1,391	45	0.183	0.266	0.315	0.403
russellmitchell	766	20	0.075	0.107	0.101	0.161
santos	1,026	21	0.067	0.110	0.183	0.171
shaw	1,222	21	0.064	0.110	0.116	0.112
wardbeck	1,076	14	0.084	0.090	0.112	0.176
wheeler	1,181	15	0.342	0.287	0.371	0.413
wilson	1,276	24	0.264	0.281	0.309	0.323

5. Which model won, and what that means

Logistic regression on rule counts and timing beats the GRU on 7 of the 8 folds, by 0.063 average precision with a 0.048 spread on that margin. It also has the best mean of the four, though that part is narrower: the deployed portable feature set is ahead of it on 3 folds. The model-against-model claim that survives the fold spread is the one about the GRU, and three things follow from it that are not all negative.

All of them find real signal. 0.251 against a 0.021 base rate is roughly twelve times better than chance, the deployed feature set at 0.220 is ten times, and the single best rule at 0.160 is not far behind either. The signal is there. The argument is only about what is needed to extract it.

The sequence model is not obviously wrong, it is just not worth it. The GRU reaches 0.188, above the best single rule, so ordering by sequence does carry information. It costs 5,458 parameters, a PyTorch dependency and a training loop to land below twenty numbers fitted by gradient descent in a fifth of a second. A fair reading is that a small recurrent model does not beat counting on this data, not that no sequence model ever could: this one had no class-balanced sampling, no per-fold threshold tuning during training and no attention over the window.

None of them is deployable as an alerting rule. Take the full logistic on its best fold, wheeler, which is 1,181 windows containing 15 attack windows. Ranked by score, it holds perfect precision down to recall

0.400: 6 attacks caught and nothing false. Push it to catch half of them, recall 0.533, and precision is 0.031, which is 8 real attacks arriving with 246 false positives beside them. That is what triage looks like at a two percent base rate, and it is the honest correction to section 7.

6. What was deployed, and why it is not the winner

The model with the best mean cannot be run on this lab, and that is worth being precise about. Its feature vector is one column per AIT rule id. This lab and AIT share two signatures out of 31. Pointed at live lab alerts, every one of them would land in the unknown column and the model would return a confident number about nothing.

So a second feature set was fitted: eleven columns that describe a window without naming a single signature. How many alerts arrived, over how long, how close together, how tightly the busiest minute was packed, how many distinct signatures were involved, and how severe they were. Severity earns its place by being a property of the Wazuh ruleset rather than of one capture, which is why it transfers where a rule id does not, and `severity-ablation.py` says what that is worth rather than leaving it as an argument. Dropping the four level columns takes the set from 0.220 to 0.191 average precision, a cost of 0.029 that shows on 7 of the 8 folds, and the four on their own reach 0.189, which is 86 % of what all eleven reach.

FEATURE	WEIGHT, STANDARDISED
busiest_60s	+1.187
n_alerts	-0.864
n_distinct_rules	+0.573
mean_level	-0.458
median_gap_s	-0.427
min_gap_s	+0.266
max_level	-0.207
n_level_ge_10	-0.200
n_level_ge_7	+0.161
max_gap_s	+0.146
duration_s	-0.143

Measured the same way as everything else, it scores **0.220 average precision (sd 0.099)**. That keeps 88 % of the full model, beats the best single rule on 8 folds of 8, and beats the base rate on 8 of 8, which is 10 times chance. The gap of 0.031 average precision against the full feature set is the measured price of portability, and it carries a spread of 0.035 across folds, so the difference between the two feature sets is smaller than the fold to fold noise on it. The portable set is ahead on 3 of 8. An earlier version of this document read that gap the other way, as evidence that most of the signal lived in *which* rules fired; that reading was an artefact of a per-window alert cap set at 256, and section 2 has what replaced it.

This is the model on the dashboard. It is fitted on 7,656 episodes from 7 networks, with its cutoff of 0.771 chosen on 1,276 episodes from wilson, which the fit never saw, where it reached precision 0.533 at recall 0.333. It runs inside the SSH poll the dashboard was already making, as a dot product in plain Python, so nothing is installed on the manager and no PyTorch goes near a box whose job is receiving alerts.

What the dashboard panel claims, and what it does not. The panel prints the model's provenance and its measured average precision underneath itself, permanently, along with the sentence that it has never been measured on this lab. It presents the score as triage ordering and not as a detection. Bars are scaled against the model's cutoff rather than against the tallest bar on screen, because auto-scaling a narrow band of scores turns a 0.03 spread into a dramatic climb, which is a picture of noise.

What the dashboard actually shows, which is not the model's number

One more step sits between the model and the panel, and it was added because the model's own output is the wrong thing to put in front of an analyst. It answers how unusual a window is. It does not answer how bad it is, and forty level 3 alerts look much like one level 12 beside a new account to something reading only shape and severity mass.

So the panel leads with a 0 to 100 severity, a composite of six terms with the model as one of them, weighted 0.28. The others are the highest rule level in the window, how much high level alerting there was rather than just the peak, how tightly the busiest minute was packed, how many distinct signatures were involved, and how many attack stages appeared. The total is multiplied by up to 1.30 when credential access and persistence both appear, because that pairing is a chain rather than two events, and a chain is exactly what six single event rules cannot see between them.

TERM	WHAT IT READS	WEIGHT
model	How far past its cutoff the model put this window	0.28
peak	The highest Wazuh rule level in the window	0.18
mass	How much high level alerting there was, not just the peak	0.16
velocity	How tightly the busiest minute was packed	0.12
breadth	How many distinct signatures were involved	0.10
coverage	How many distinct attack stages appeared	0.16

On a fabricated run used to test the path end to end, a quiet window of session opens scores 22, a brute force burst on its own scores 61, and the same burst with an account creation beside it scores 86. Step 4 cannot separate the last two: it reports a brute force alert and an account creation alert, and nothing that says they belong together.

Every constant in that composite is judgement rather than a fitted parameter, and the panel says so. They are in one file, `scoring/scorer/score.py`, and the panel prints the weights it was actually given rather than a copy, so the explanation on screen cannot drift from the arithmetic behind it. A window reaching 50 becomes a finding, and findings export to their own PDF.

7. The synthetic results, and why they were discarded

Before the public data, the pipeline was developed against a generator, `make-synthetic.py`, which produces 167 episodes with 56 attacks in them. Re-run for this report, the numbers are:

MODEL, ON SYNTHETIC DATA	F1	AVERAGE PRECISION
Rule 100111 alone	0.857	—
GRU over the sequence, 5 seeds	0.948 (sd 0.018)	0.997
Logistic on counts and timing	1.000	1.000

Everything near the ceiling, and none of it means anything. The generator builds in exactly two signals, that an attacker brute forces before persisting and moves between steps in seconds, and both are linearly separable from counts and a burst measure. A good score there was evidence that the pipeline ran, not that anything had been detected. Real alerts cost the GRU 0.784 of its f1.

Reporting this is the point of including it. A model evaluated only on data produced by assumptions it then recovers will always look excellent, and the difference between 0.997 and 0.188 is the entire distance between a demo and a measurement.

8. What this does not support

- **It is not a test of this lab's detections.** Rules 100100 to 100113 appear in no public dataset. Nothing here says whether they separate an attacker from an administrator in sequence.
- **It is not an alerting capability.** At this base rate the best operating point found catches a fifth of intrusions at perfect precision, or drowns the queue. The deployed model is ordering, not detection.
- **The deployed model has never been measured on this network.** Its 0.220 is what it scored on networks it had not seen. This lab is a ninth such network, so that is the honest expectation for it and not a result from it.
- **The GRU has not been given every chance.** The conclusion is that a small recurrent model does not beat counting here, on this data, at this scale.
- **Windows scenarios are untouched.** Rules 100100 to 100103 are unreachable from a Linux campaign.

9. What would change the answer

A campaign on the lab itself. `run-campaign.sh` drives the three scenarios against both endpoints alongside three standing accounts with deliberately different working shapes, and produces alerts carrying this lab's own rule ids with ground truth labels attached. That converts three of the five limitations above into measurements: whether the lab's rules separate in sequence, whether a per-user pattern of life helps, and what the deployed model actually scores here rather than elsewhere.

It was trialled successfully for one hour on 17 September. The full run is 14 hours. It is now a specific question rather than a blocker, which is the main thing the public data bought.